



UNICA

UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI



sAifer Lab

Joint lab on Safety and Security of AI

Frontend

Angelo Sotgiu

angelo.sotgiu@unica.it

In questa lezione

- Rendering HTML
- Templating
- Form



Riprendiamo il nostro sistema biblioteca...

- Finora abbiamo sviluppato diverse API, lavorando sul lato **backend** dell'applicazione web
- Al momento, possiamo interagire con gli endpoint dell'app soltanto eseguendo direttamente richieste HTTP
- Lavoreremo ora sul lato **frontend**, fornendo una semplice interfaccia tramite la quale interagire con l'applicazione

books

GET	/books/	Get All Books
POST	/books/	Add Book
DELETE	/books/	Delete All Books
GET	/books/{id}	Get Book By Id
PUT	/books/{id}	Update Book
DELETE	/books/{id}	Delete Book
POST	/books/{id}/review	Add Review

Un piccolo test

- Proviamo a scrivere un semplice HTML e darlo in risposta dalla nostra applicazione web. Scriviamo un endpoint sulla root (file main.py) che risponde a richieste HTTP GET:

```
@app.get("/")
def home():
    html = """
    <!DOCTYPE html>
    <html>
        <body>
            <h1>Hello world!</h1>
            <p>Hellooo!</p>
        </body>
    </html>
    """
    return html
```

Un piccolo test

- Cosa succede? <http://127.0.0.1:8000/>
Non funziona come dovrebbe! Perché?
- Analizzando la risposta HTTP, notiamo che il body contiene il nostro HTML, ma l'header Content-Type è application/json
- Questo perché di default FastAPI converte i dati che restituiamo in JSON!
Il browser quindi non è in grado di renderizzare correttamente l'HTML.
- Dobbiamo esplicitamente dire a FastAPI che la nostra risposta è in formato HTML:

```
from fastapi.responses import HTMLResponse
```

```
@app.get("/", response_class=HTMLResponse)
```

File HTML e Templating

- Lavorare sul nostro HTML come stringa in Python è un po' scomodo...
- Spostiamo tutto su un file:
 - creiamo una cartella **templates** dentro app
 - dentro templates, creiamo un file "home.html" e copiamo dentro il nostro HTML
- Per caricare e renderizzare i file HTML, sfrutteremo la funzionalità detta **templating**
- Vedremo nella pratica il significato di questo termine...
- ...per adesso ci basta sapere che è un modo per generare dinamicamente pagine web a partire da file HTML statici
- Esistono diversi tool per templating, useremo Jinja2 (già integrato in FastAPI)
<https://jinja.palletsprojects.com/en/stable/templates/>



Templating

- Modifichiamo in questo modo la nostra funzione endpoint :

```
from fastapi import FastAPI, Request
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates
from routers import books

app = FastAPI()
app.include_router(books.router, tags=["books"])
templates = Jinja2Templates(directory="app/templates")

@app.get("/", response_class=HTMLResponse)
def home(request: Request):
    return templates.TemplateResponse(
        request=request, name="home.html"
    )
```



Frontend

- Iniziamo ora a scoprire i vantaggi del templating...
- ...ma prima, riorganizziamo il codice:
 - creiamo un nuovo file "frontend.py" dentro il package "routers"
 - inizializziamo un nuovo router, questa volta per il percorso root (quindi nessun parametro prefix)
`router = APIRouter()`
 - spostiamo la funzione di endpoint dal main.py dentro il nuovo file;
ricordiamoci di sostituire il decoratore `@app.get` con `@router.get`
 - nel main.py, importiamo il nuovo router e includiamolo nell'app
`from fastapi import FastAPI`
`from routers import frontend, books`

```
app = FastAPI()
app.include_router(frontend.router)
app.include_router(books.router, tags=["books"])
```


Templating - Estensione

- Sfruttiamo la funzionalità di estensione dei template per scrivere in un unico HTML con le porzioni dell'interfaccia comuni a tutte le pagine web, in modo da non doverle riscrivere ogni volta
- Creiamo dentro la cartella "template" un file "base.html" con il contenuto della slide seguente
- La parte importante è questa:
`{% block content %}{% endblock %}`
- Si tratta di un placeholder, in questo caso con id "content", che può essere "riempito" dinamicamente con un altro html
- Header, barra di navigazione e footer saranno sempre mostrate nelle pagine che estenderanno "base.html"

```
<!DOCTYPE html>
<html>
  <head>
    <title>Library</title>
  </head>
  <body>
    <header>
      <h1>Library</h1>
      <p>A simple web application example.</p>
    </header>
    <nav>
      <a href="/">Home</a>
      <a href="#">Book List</a>
      <a href="#">Add Book</a>
    </nav>
    <div class="container">
      {% block content %}{% endblock %}
    </div>
    <footer>
      <p>&copy; 2025 Programmazione Web UNICA </p>
    </footer>
  </body>
</html>
```



Templating - Estensione

- Modifichiamo adesso il file "home.html" in modo da estendere "base.html". Dovrà contenere all'inizio questa riga:

```
{% extends "base.html" %}
```

- ...e il contenuto aggiuntivo all'interno del blocco desiderato

```
{% block content %}
```

```
...
```

```
{% endblock %}
```

```
{% extends "base.html" %}
```

```
{% block content %}
```

```
    <h3 style="text-align: center">Welcome to the library</h3>
```

```
{% endblock %}
```

Templating - Variabili

- Possiamo anche "passare" dinamicamente dati dal backend al frontend!
Vediamo subito un piccolo esempio
- Modifichiamo "home.html" così

```
{% extends "base.html" %}  
{% block content %}  
    <h3 style="text-align: center">{{ text }}</h3>  
{% endblock %}
```

- Le doppie parentesi graffe segnalano la presenza di una variabile "text", che dovrà essere fornita dal backend.
Il motore di templating si occuperà di sostituire la variabile con il contenuto ricevuto, e renderizzare la pagina web.

Templating - Variabili

- Dal backend, dovremo fornire un dizionario "context" che contiene il nome delle variabili attese dal template come chiavi e il loro contenuto come valori.
Nel nostro esempio:

```
@app.get("/", response_class=HTMLResponse)
def home(request: Request):
    return templates.TemplateResponse(
        request=request, name="home.html",
        context={"text": "Welcome to the library"}
    )
```

- Verifichiamo il corretto funzionamento <http://127.0.0.1:8000/>



Templating - Escaping

- ATTENZIONE! Dato che le variabili fornite nel contesto sono automaticamente trasformate in stringhe e messe dentro l'HTML, se il contenuto delle variabili è esso stesso HTML verrà interpretato come tale!
- Questo può avere effetti indesiderati, e soprattutto creare vulnerabilità se un'attaccante è in grado di alterare di proposito questi dati!
Ad esempio, inserendo degli script malevoli, che verrebbero poi eseguiti.
- Per evitare questo, dobbiamo sanitizzare i dati con l'operazione di escaping (ovvero, i caratteri speciali come <, >, &, etc. saranno codificati come normali stringhe e non come codice HTML.
- Di default, Jinja2 non esegue automaticamente l'escaping per ragioni computazionali. Viene consigliato di eseguirlo manualmente con il filtro `|e` da aggiungere solo dopo le variabili considerate a rischio.

```
<p>{{ text|e }}</p>
```

Templating - Variabili con attributi

- Possiamo fornire qualsiasi tipo di dato nel contesto, verrà convertito in stringa.
- Inoltre, è possibile accedere ai suoi attributi direttamente nel template. Esempio (file "home.html", al posto dell'h3):

```
<div style="text-align: center">  
  <h1>{{ text.title }}</h1>  
  <p>{{ text.content }}</p>  
</div>
```

```
@app.get("/", response_class=HTMLResponse)  
def home(request: Request):  
    text = {  
        "title": "Welcome to the library",  
        "content": "Have you ever seen this place?"  
    }  
    return templates.TemplateResponse(  
        request=request, name="home.html", context={"text": text}  
    )
```

Templating – Operazioni avanzate

- Tramite la seguente sintassi possiamo sfruttare diverse funzionalità avanzate di Jinja2
`{% ... %}` -> al posto dei puntini inseriremo il costrutto necessario
- Esempio: ciclo `for`

```
{% for word in sequence %}  
    <p>{{ word }}</p>  
{% endfor %}
```

- Nella funzione endpoint dovremmo aggiungere nel context:

```
context={"text": text, "sequence": ["a", "b", "c"]}
```


Templating – Operazioni avanzate

- La sintassi dentro il for è simile a quella di Python.
Ad esempio, sui dizionari possiamo iterare su chiavi e valori, oppure solo sulle chiavi:

```
{% for key, value in dictionary.items() %}  
  <p>Key: {{ key }} Value: {{ value }}</p>  
{% endfor %}
```

oppure

```
{% for key in dictionary %}  
  <p>Key: {{ key }} Value: {{ text[key] }}</p>  
{% endfor %}
```

- Nella funzione endpoint dovremo aggiungere nel context:

```
context={"text": text, "dictionary": {"a": 0, "b": 1, "c": 2}}
```

FastAPI – File statici

- Vogliamo adesso aggiungere un'immagine nella homepage. I file statici (CSS, immagini, etc.) dovrebbero essere contenuti in un'apposita cartella:
 - creiamo una cartella "static" dentro "app"
 - copiamoci dentro un'immagine qualsiasi
- Dobbiamo adesso configurare FastAPI in modo che monti la cartella static. Aggiungiamo nel main.py il seguente import:

```
from fastapi.staticfiles import StaticFiles
```

e questo comando dopo la creazione dell'oggetto app:

```
app.mount("/static", StaticFiles(directory="app/static"), name="static")
```

FastAPI – File statici

- Possiamo adesso modificare il file "home.html" in questo modo:

```
{% extends "base.html" %}
{% block content %}
    <h3 style="text-align: center">{{ text }}</h3>
    <br>
    <div style="text-align: center">
        
    </div>
{% endblock %}
```

- Dentro le doppie parentesi graffe possiamo anche chiamare delle funzioni.
In questo caso stiamo chiamando il metodo "url_for", che recupera il percorso di una risorsa esposta dalla nostra applicazione web, ovvero la cartella "/static", e il file indicato nel parametro "path".

File CSS

- L'interfaccia fa un po' schifo... proviamo ad aggiungere un file CSS
 - creiamo un file "styles.css" al suo interno
 - riempiamolo con qualche semplice configurazione (continua nella prossima slide)

```
* {  
    margin: 0;  
    padding: 0;  
    box-sizing: border-box;  
}  
  
body {  
    font-family: Arial, sans-serif;  
    background-color: #f2f2f2;  
    color: #333;  
    line-height: 1.6;  
}
```

- Qualche altra configurazione...

```
nav {  
  background: #005599;  
  display: flex;  
  justify-content: center;  
  padding: 10px;  
}  
nav a {  
  color: #fff;  
  text-decoration: none;  
  margin: 0 15px;  
  font-size: 16px;  
  transition: color 0.3s;  
}  
  
nav a:hover {  
  color: #ffcc00;  
}  
  
.container {  
  max-width: 1200px;  
  margin: 20px auto;  
  padding: 20px;  
  background-color: #fff;  
}  
  
header {  
  background: #003366;  
  color: #fff;  
  padding: 20px;  
  text-align: center;  
}  
  
footer {  
  background: #003366;  
  color: #fff;  
  text-align: center;  
  padding: 15px;  
  margin-top: 20px;  
}
```

- Infine, aggiungiamo il link al file CSS nel file "base.html":

```
<head>  
  <title>Library</title>  
  <link rel="stylesheet" href="{{ url_for('static', path='/styles.css') }}">  
</head>
```



Pagina con elenco libri

- Creiamo adesso un template "list.html" per mostrare la lista dei libri in una tabella, costruita dinamicamente:

```
{% extends "base.html" %}
{% block content %}
<table>
  <tr>
    <th>ID</th>
    <th>Title</th>
    <th>Author</th>
    <th>Review</th>
  </tr>
  {% for book in books %}
    <tr>
      <td>{{ book.id }}</td>
      <td>{{ book.title }}</td>
      <td>{{ book.author }}</td>
      <td>{{ book.review }}</td>
    </tr>
  {% endfor %}
</table>
{% endblock %}
```

Pagina con elenco libri

- Creiamo un'altra funzione endpoint che renderizzi il template passando la lista dei libri:

```
from fastapi import APIRouter, Request
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates
from data.books import books

router = APIRouter()
templates = Jinja2Templates(directory="app/templates")

@router.get("/book_list", response_class=HTMLResponse)
def home(request: Request):
    context = {"books": list(books.values())}
    return templates.TemplateResponse(
        request=request, name="list.html", context=context
    )
```

Pagina con elenco libri

- Nel CSS aggiungiamo...

```
table {  
    width: 50%;  
    margin: 0 auto;  
    border-collapse: collapse;  
}
```

```
th, td {  
    padding: 10px;  
    border: 1px solid #ddd;  
}
```

```
th {  
    background-color: #f5f5f5;  
}
```

- Aggiungiamo anche un link alla nuova pagina nella barra di navigazione, nel file "base.html":

```
<a href="{{ url_for('show_book_list') }}">Book List</a>
```


Pagina per l'aggiunta di un libro

- Creiamo un nuovo template "add.html" per l'aggiunta di un nuovo libro. Useremo i **form**.

```
{% extends "base.html" %}
{% block content %}
    <form>
        <label for="id">ID:</label><br>
        <input type="number" name="id" id="id" min="0" required><br>
        <label for="title">Title:</label><br>
        <input type="text" name="title" id="title" required><br>
        <label for="author">Author:</label><br>
        <input type="text" name="author" id="author" required><br>
        <label for="review">Review:</label><br>
        <input type="number" name="review" id="review" min="1" max="5" required>
        <br><br>
        <input type="submit" value="Add">
        <input type="reset" value="Reset">
    </form>
{% endblock %}
```

Pagina per l'aggiunta di un libro

- Creiamo un'altra funzione endpoint che renderizzi il template:

```
@router.get("/add_book", response_class=HTMLResponse)
def add_book_form(request: Request):
    return templates.TemplateResponse(
        request=request, name="add.html"
    )
```

- Problema: di default, facendo "submit", il form HTML:
 - esegue una post allo stesso URL in cui è mostrato il form
 - passa gli argomenti nel body, ma codificati come query string
- Dobbiamo:
 - modificare l'URL al quale il form esegue la POST
 - scrivere una nuova funzione endpoint che possa leggere correttamente il body prodotto dal form, perché l'API "/books" si aspetta un JSON

Pagina per l'aggiunta di un libro

- Nel file "routers/books.py" aggiungiamo una nuova funzione endpoint all'url "/books_form":

```
from fastapi import APIRouter, Path, HTTPException, Query, Form

@router.post("_form/")
def add_book_from_form(book: Annotated[Book, Form()]):
    """Adds a new book."""
    if book.id in books:
        raise HTTPException(status_code=403, detail="Book ID already exists")
    books[book.id] = book
    return "Book successfully added"
```

- Modifichiamo così il comportamento del submit del form:

```
<form action="{ url_for('add_book_from_form') }" method="post">
```

Pagina per l'aggiunta di un libro

- Nel CSS aggiungiamo...

```
input[type=submit], input[type=reset] {  
    font-size: 16px;  
    background: #005599;  
    border: none;  
    color: white;  
    padding: 16px 32px;  
    cursor: pointer;  
}
```

- Aggiungiamo anche un link alla nuova pagina nella barra di navigazione, nel file "base.html":

```
<a href="{ { url_for('add_book_form') } }">Add Book</a>
```